

Ejercicio 3

Investigación REACT

¿Qué es JSX en React ?

JSX (*JavaScript XML*) es una extensión de sintaxis de JavaScript que permite escribir código con aspecto de HTML dentro de JavaScript. Sirve para describir de forma clara cómo debe verse la interfaz.

¿Cuáles son sus características y para qué sirve?

- ☆ Se parece a HTML, pero es JavaScript: internamente se transforma en llamadas a `React.createElement()`.
- ☆ Permite insertar expresiones de JavaScript entre llaves `{}`.
- ☆ Un componente debe devolver un único elemento raíz (o un fragmento `<> </>`).
- ☆ Usa `className` en lugar de `class`, y los atributos van en camelCase (`onClick`).
- ☆ Todas las etiquetas deben cerrarse (`<img />`).

Ejemplo de implementación:

```
1  function Saludo() {  
2    const nombre = "Jane";  
3    const anio = 2026;  
4  
5    return (  
6      <div>  
7        <h1>Hola, {nombre}</h1>  
8        <p>El año actual es {anio}</p>  
9      </div>  
10   );  
11 }  
12  
13 export default Saludo;  
14
```

Explicación: el componente Saludo define dos variables y las muestra dentro del JSX usando llaves. Al renderizarse, se ve "Hola, Jane" y "El año actual es 2026". Sin JSX habría que armar cada elemento con `React.createElement`.

¿Qué son los componentes en React?

Los componentes son piezas independientes y reutilizables de la interfaz. Cada uno se encarga de una parte (un botón, un menú, una tarjeta) y devuelve JSX. Sirven para dividir una interfaz compleja en partes pequeñas, más fáciles de mantener.

¿Cuáles son sus características y para qué sirven?

- ☆ Son reutilizables: se escriben una vez y se usan las veces que haga falta.
- ☆ Son independientes, cada uno tiene su propia lógica y apariencia.
- ☆ Pueden ser de función (lo más usado hoy) o de clase.
- ☆ Su nombre siempre empieza con mayúscula.
- ☆ Se pueden anidar unos dentro de otros.

Ejemplo de implementación:

```
1  function Tarjeta() {  
2    return (  
3      <div className="tarjeta">  
4        <h2>Mi tarjeta</h2>  
5        <p>Este es un componente reutilizable.</p>  
6      </div>  
7    );  
8  }  
9  
10 function App() {  
11   return (  
12     <div>  
13       <Tarjeta />  
14       <Tarjeta />  
15     </div>  
16   );  
17 }  
18  
19 export default App;  
20
```

Explicación: Tarjeta es un componente que devuelve un bloque de JSX. App lo utiliza dos veces con la etiqueta <Tarjeta />, y en pantalla aparecen dos tarjetas idénticas sin repetir código.

¿Qué son los props en React?

Los props (*properties*) son los datos que un componente padre le pasa a un componente hijo. Sirven para personalizar un mismo componente con distintos valores.

¿Cuáles son sus características y para qué sirven?

- ☆ Se pasan como atributos en el JSX: <Tarjeta titulo="React" />.
- ☆ Son de solo lectura: el componente hijo no puede modificarlos.
- ☆ Los datos fluyen en una sola dirección, de padre a hijo.
- ☆ Pueden ser textos, números, booleanos, objetos, arrays o funciones.

Ejemplo de implementación:

```
1  function Tarjeta(props) {
2    return (
3      <div className="tarjeta">
4        <h2>{props.titulo}</h2>
5        <p>{props.descripcion}</p>
6      </div>
7    );
8  }
9
10 function App() {
11   return (
12     <div>
13       <Tarjeta titulo="React" descripcion="Biblioteca para crear interfaces." />
14       <Tarjeta titulo="Firebase" descripcion="Plataforma de Google para apps." />
15     </div>
16   );
17 }
18
19 export default App;
20
```

Explicación: App usa dos veces el componente Tarjeta, pero con distintos valores en titulo y descripcion. Tarjeta los recibe en props y los muestra, así que el mismo componente muestra contenido diferente en cada uso.

¿Qué es Firebase?

Firebase es una plataforma de Google para desarrollar aplicaciones web y móviles. Ofrece servicios de backend listos para usar (base de datos, autenticación, hosting, etc.), por lo que no hace falta programar ni mantener un servidor propio.

¿Cuáles son sus características ?

- ☆ **Authentication:** registro e inicio de sesión de usuarios (correo y contraseña, Google, etc.).
- ☆ **Cloud Firestore / Realtime Database:** bases de datos NoSQL en la nube, con sincronización en tiempo real.
- ☆ **Hosting:** publicación de aplicaciones web con certificado HTTPS.
- ☆ **Cloud Storage:** almacenamiento de archivos como imágenes o documentos.
- ☆ **Cloud Functions:** ejecución de código en el servidor sin administrarlo.
- ☆ Cuenta con SDK para JavaScript, por lo que se integra fácilmente con React.
- ☆ Tiene un plan gratuito, útil para proyectos chicos y de aprendizaje.